

A KEYHAVEN FIELD GUIDE

Tokens, SSH, Signing & CI

How GitHub knows it's you, how your app's updates stay trustworthy,
and how the robots build and ship your releases.

Written for the Keyhaven project · keep this for when you fly solo.

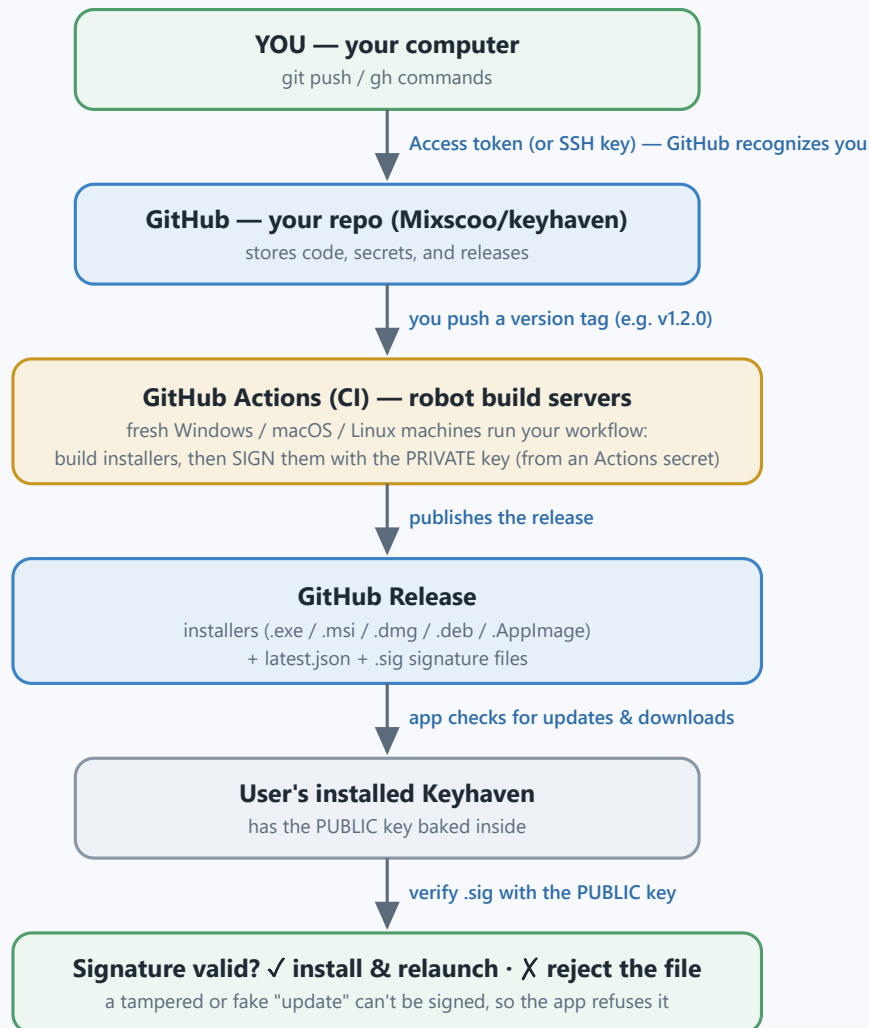
Contents

1. The big picture — four things, two jobs
2. Authentication 1: Personal Access Tokens
3. Authentication 2: SSH keys
4. Signing keys — trustworthy app updates
5. CI/CD — the robot that builds and ships
6. Do-it-yourself: release checklist
7. Glossary & cheat sheet

Tip: this is a printable e-book. Open it in a browser and choose **Print** → **Save as PDF** if you ever need a fresh copy.

Keyhaven — Identity, Signing & CI at a glance

How you reach GitHub, how a release is built & signed, and how users get safe updates



The three "keys/secrets" — don't mix them up

- **Access token / SSH key** proves YOU to GitHub · lives on your PC
- **Signing PRIVATE key** signs updates · GitHub Actions secret + your backup
- **Signing PUBLIC key** verifies updates · baked into every installed app
- **CI** the robot that builds, signs & publishes on a version tag

The whole flow on one page — keep this handy while the chapters explain each step.

1 · The big picture — four things, two jobs

Everything that confused you boils down to **four pieces** doing **two kinds of job**. Get this map straight and the rest clicks.

The two jobs

- **Authentication** — proving *who you are*. ("GitHub, it's me, Mixscoo.")
- **Signing** — proving *a file is genuine and untampered*. ("User's app: this update really came from Keyhaven.")

They use different keys, aimed at different audiences. People mix them up constantly — you won't anymore.

The four pieces

Piece	Its one job	Audience	Lives where
Access token (PAT)	proves YOU	GitHub	your PC (keyring)
SSH key	proves YOU (alt. to token)	GitHub	your PC (<code>~/.ssh</code>)
Signing key (private+public)	proves an update is genuine	users' apps	CI secret + your backup / baked in app
CI (GitHub Actions)	builds, signs & ships automatically	—	GitHub's servers

One sentence to remember: a *token/SSH key* proves you to GitHub; a *signing key* proves your updates to users; *CI* is the robot that ties it together on a version tag.

2 · Authentication 1 — Personal Access Tokens

A **Personal Access Token (PAT)** is a long, random secret string that works as a password replacement. GitHub turned off real-password git access in 2021, so you authenticate with a token (or SSH, next chapter).

Why a token beats a password

- **Scoped** — you choose what it can do (e.g. only touch one repo's contents).
- **Revocable** — delete it anytime without changing your account password.
- **Expirable** — you can give it a lifetime.

Create one by hand

1. github.com → your avatar → **Settings**
2. Bottom-left → **Developer settings**
3. **Personal access tokens** → **Fine-grained tokens** → **Generate new token**
4. Pick a name, expiry, the repo(s) it may access, and permissions (e.g. *Contents: Read and write*)
5. **Generate**, then **copy it immediately** — GitHub shows it once and never again.

How it's used

When you `git push` over an `https://` remote, git sends the token as the "password." A **credential helper** caches it so you don't paste it every time. Think of it as swiping an ID badge: GitHub looks up the token, sees it maps to your account with these permissions, and allows the action.

The easy way (what you actually did)

You didn't hand-make a PAT — you ran the GitHub CLI:

```
gh auth login      # opens a browser login, stores a token for you
gh auth status     # shows the account + token it's using
```

That stored a token (`gho_...`) in your OS **keyring**. Same concept, friendlier setup.

Never paste a token into code, screenshots, or chats. If one leaks, revoke it in Settings → Developer settings and generate a new one.

3 · Authentication 2 — SSH keys

SSH is the other way to prove you're you. Instead of sending a token, your machine *proves* it holds a secret key — without ever transmitting it.

The key pair

- **Private key** — stays on your machine, never shared.
- **Public key** — you upload it to GitHub; it's safe to share.

How the handshake works

When you connect to `git@github.com`, GitHub issues a cryptographic challenge. Your private key answers it; GitHub checks the answer against your uploaded public key. If they match, you're in — no token typed.

Set it up

1. Generate a pair:

```
ssh-keygen -t ed25519 -C "mgb12203@gmail.com"
```

This makes `~/.ssh/id_ed25519` (private — keep secret) and `id_ed25519.pub` (public).

2. Copy the contents of the `.pub` file.
3. GitHub → Settings → **SSH and GPG keys** → **New SSH key** → paste.
4. Use SSH remote URLs, e.g. `git@github.com:Mixscoo/keyhaven.git`.

Token vs SSH — pick one

	Token / HTTPS	SSH
Setup	simplest (esp. with <code>gh</code>)	generate + upload key
Expiry to manage	often yes	no
Best when	occasional use, CLI tooling	you push constantly

You're on **token + HTTPS** already. That's perfectly good — you don't need SSH unless you prefer it.

4 · Signing keys — trustworthy app updates

This key has **nothing to do with GitHub identity**. Its audience is your *users' installed apps*. Because Keyhaven auto-downloads updates from the internet, each app must be sure an update is really yours and untampered — otherwise a poisoned "update" could steal vaults. Signing guarantees authenticity.

The wax-seal analogy

Only you own the signet ring (the **private key**) that stamps each release. Everyone recognizes your seal (the **public key**) but can't forge it. A release with a valid seal is trusted; a missing/wrong seal is rejected.

Create the key

```
npm run tauri signer generate -w keyhaven-updater.key --ci
```

This produces `keyhaven-updater.key` (private) and `keyhaven-updater.key.pub` (public).

Set it up

1. **Public key** → paste into `src-tauri/tauri.conf.json` under `plugins.updater.pubkey`. It gets baked into every build.
2. **Private key** → store as a GitHub **Actions secret** so CI can sign:

```
Get-Content keyhaven-updater.key -Raw | gh secret set TAURI_SIGNING_PRIVATE_KEY
```

(Or repo → Settings → Secrets and variables → Actions → New repository secret.)

3. Also set `TAURI_SIGNING_PRIVATE_KEY_PASSWORD` (empty if the key has no passphrase).
4. **Back up** `keyhaven-updater.key` somewhere safe and never commit it.

How it flows

CI signs each installer with the private key → produces a `.sig` + a `latest.json` → uploads them to the release. A user's app downloads the update and its `.sig`, then verifies it with the embedded public key. Valid → install; invalid → reject.

Guard this key with your life. The public half is permanently embedded in every installed copy. If you lose the private key and generate a new one, all existing installs will *reject* future updates (the seal no longer matches). Back it up.

A local `npm run tauri build` that says "*no private key*" is normal — your laptop doesn't hold the signing key. The installers still build fine; signing happens in CI. To sign locally, set `TAURI_SIGNING_PRIVATE_KEY` from your key file first.

5 · CI/CD — the robot that builds and ships

CI = Continuous Integration. In plain terms: **robot computers on GitHub's servers that automatically run your commands when something happens** (a push, a tag, a pull request). **CI/CD** adds "Continuous Delivery" — it also publishes the result (your installers).

Why bother

- Builds on clean machines — no "works on my laptop" surprises.
- Builds **Windows, macOS, and Linux at once** — your one PC can't.
- Signs and publishes automatically, so a release is one command.

GitHub Actions vocabulary

Term	Meaning
Workflow	a YAML file in <code>.github/workflows/</code> describing the steps (yours: <code>release.yml</code>)
Trigger (<code>on:</code>)	what starts it — yours: pushing a tag like <code>v1.2.0</code>
Runner	the throwaway virtual machine GitHub boots
Job	a group of steps on a runner (yours repeats across 4 OS targets via a "matrix")
Step	one command or action (checkout, install, build...)
Secret	an encrypted value injected as an env var, masked in logs (your signing key)

What a run does (your `release.yml`, end to end)

1. You push tag `v1.2.0` → GitHub sees the `on: push: tags: v*` trigger.
2. It boots fresh VMs (Windows, macOS ×2, Linux).
3. Each runs: checkout → install Node → install Rust → install deps → **build** → **sign** (using the secret) → **create the Release** and upload installers + `latest.json` .
4. A temporary `GITHUB_TOKEN` (auto-created per run, expires at the end) lets the workflow publish on your behalf.
5. Green checkmarks; the release appears with downloads.

Two different "GitHub tokens": your `gho_...` token authenticates *you* from your PC; the workflow's `GITHUB_TOKEN` is a short-lived one GitHub mints for each CI run so the robot can act on the repo.

How you use / change it

- **Use:** `npm run release 1.2.0` (pushes the tag → triggers CI).
- **Watch:** `gh run watch` or the repo's **Actions** tab.

- **Change:** edit `.github/workflows/release.yml` .

6 · Do-it-yourself — release checklist

When you want to ship an update entirely on your own:

1. **Make the changes** (code/features/fixes).
2. **Verify locally:** `npm run check` and `npm test` (and optionally `npm run tauri build` to smoke-test).
3. **Commit & push** the changes to the repo.
4. **Pick the version** with SemVer:
 - bug fix → patch (1.1.0 → 1.1.1)
 - new feature → minor (1.1.0 → 1.2.0)
 - breaking change → major (1.x → 2.0.0)
5. **Release:** `npm run release 1.2.0` — bumps every manifest, commits, tags, pushes; the guard refuses anything not higher than the current version.
6. **Done.** CI builds, signs, and publishes. Users on the prior signed version auto-update on next launch.

The version you type is the *only* place it lives — the script writes it into all manifests and the in-app Settings display follows automatically. Never hand-edit version numbers.

Only versions $\geq$ **the first signed release** auto-update. Anyone on an older build installs the new one manually once, then they're on the auto-update train.

7 · Glossary & cheat sheet

Term	In one line
Authentication	proving <i>who you are</i>
Signing	proving a <i>file is genuine & untampered</i>
PAT / access token	secret string that proves you to GitHub over HTTPS
SSH key	key pair that proves you to GitHub over SSH
Private signing key	signs updates; secret; CI + your backup
Public signing key	verifies updates; baked into the app
Actions secret	encrypted value stored in the repo, used by CI
<code>GITHUB_TOKEN</code>	temporary per-run token CI uses to publish
CI / CD	robot that builds (CI) and ships (CD) automatically
Workflow	YAML recipe in <code>.github/workflows/</code>
Runner	throwaway VM that runs a workflow
SemVer	MAJOR.MINOR.PATCH versioning
<code>latest.json</code>	file the updater reads to learn the newest version
<code>.sig</code>	signature proving an installer is genuine

Command cheat sheet

```
gh auth status          # who am I to GitHub?
git push                # uses your token/SSH to authenticate
npm run check && npm test # verify before releasing
npm run release 1.2.0   # bump + commit + tag + push (triggers CI)
gh run watch            # follow the CI build
gh run list --workflow=release.yml
```

The whole story in one breath: you authenticate to GitHub with a token (or SSH); you push a version tag; CI builds your app on clean machines and signs it with your private key; GitHub publishes the release; users' apps verify the signature with the baked-in public key and update themselves. Four pieces, two jobs, one smooth pipeline.